

---

# R3.06 - TP 1

## Sockets

### Rappels

#### Ordinateur au sens large

Un ordinateur n'est pas uniquement un PC<sup>1</sup>.

On trouve des ordinateurs partout autour de nous et certains ont des formes inattendues voire insoupçonnées. Un smartphone, une montre connectée, une caméra de surveillance, une TV, un distributeur de café sont des appareils qui peuvent posséder un OS et c'est souvent un Linux modifié et adapté pour la fonction spécifique de l'équipement. Dans ce cas, ce sont donc des ordinateurs au même titre qu'un PC sur un bureau.

On va parler d'ordinateur dans la suite de ce TP et les TP suivants, mais il faut le prendre au sens large du terme : tout équipement électronique qui intègre un OS.

#### Adresse IP

Un ordinateur possède une **adresse IP** s'il est relié à un réseau Ethernet ou un réseau wifi, ce qui est aujourd'hui le cas de la quasi-totalité des ordinateurs.

Comme une adresse postale, une adresse IP est unique et permet d'identifier et de localiser chaque ordinateur sur le réseau.

L'unicité de l'adresse IP est soit globale, soit locale.

Il existe des **adresses IP publiques**, celles des machines connectées directement à Internet (souvent des serveurs), et dans ce cas l'unicité est globale. Un seul ordinateur dans le monde peut posséder une adresse IP publique donnée. Ces machines font partie du réseau Internet global.

Il existe aussi des **adresses IP privées** pour les machines dans les entreprises et chez les particuliers mais qui sont situées derrière un routeur ou une box Internet par exemple. Ces machines accèdent à Internet mais ne font pas partie du réseau Internet lui-même, c'est le routeur ou la box Internet qui sert de passerelle et qui fait partie du réseau global.

---

<sup>1</sup> **PC** : **P**ersonal **C**omputer, un ordinateur personnel. Un Mac est aussi un "PC" même si on entend plutôt PC-Windows derrière le terme PC.

Il existe des plages d'adresses privées en fonction de la classe d'adressage. Sans entrer dans les détails, une adresse privée sera entre **10.0.0.0** et **10.255.255.255** ou entre **172.16.0.0** et **172.31.255.255** ou encore entre **192.168.0.0** et **192.168.255.255**

## Protocoles

Un réseau fonctionne à l'aide de protocoles.

Un protocole est un ensemble de règles de dialogue entre les équipements reliés au réseau.

Généralement un protocole est assez simple : un équipement<sup>2</sup> envoie une requête sur le réseau, à destination d'un autre équipement, et attend une réponse. Quand la réponse est reçue, en fonction de la nature de la réponse, un nouvel échange peut avoir lieu sous la même forme d'une requête+réponse. Il en va ainsi jusqu'à ce que l'un des deux interlocuteurs décide de mettre un terme à l'échange.

Le réseau Internet repose sur le protocole **TCP/IP**. Il s'agit en fait de 2 protocoles qui travaillent conjointement : **T**ransfer **C**ontrol **P**rotocol et **I**nternet **P**rotocol.

Astuce : les acronymes des protocoles finissent généralement par un **P** pour **P**rotocol.

## Services réseau

Un service est un logiciel tournant sur un ordinateur et qui n'est attaché à aucun utilisateur humain particulier. Il s'exécute sans intervention humaine, en arrière-plan. Il est généralement démarré au lancement de l'OS et s'arrête avec lui avant l'extinction de l'ordinateur.

Les services tirent leur nom du rôle qu'ils ont : ils rendent des services aux utilisateurs. Il peut s'agir par exemple d'un service de messagerie (pas le logiciel de lecture de vos mails, mais celui qui aiguille le trafic, une sorte de gare de triage de mails), d'un service Web (qui délivre des pages et des applications Web) ou encore d'un service d'impression ou de stockage de fichiers.

On appelle aussi ces services des serveurs, au sens logiciel du terme. On dit par exemple un serveur Web ou un serveur de messagerie, même si on devrait plutôt dire un service Web et un service de messagerie.

La terminologie Unix les nomme aussi des **daemons**<sup>3</sup> pour refléter une caractéristique importante qui est leur capacité à résister à leur mort. La façon spéciale dont on code un daemon fait que s'il arrivait à planter, il serait automatiquement relancé, comme un phœnix renaît de ses cendres.

Même si les services sont souvent associés à des serveurs (ordinateur), les ordinateurs personnels ont aussi des services en fonctionnement dès le démarrage de l'OS.

---

<sup>2</sup> Il peut s'agir d'un ordinateur mais aussi d'un équipement gérant le trafic réseau, tel qu'un routeur ou une box Internet.

<sup>3</sup> A ne pas traduire par démon, mais qui vient d'un mot grec signifiant "divinité"

Beaucoup de services sont liés au réseau. On les nomme alors des services réseau : serveur Web, serveur de messagerie, serveur FTP, serveur SSH, etc.

Un service utilise un (parfois plusieurs) protocoles pour fournir son service aux autres équipements qui en ont besoin. Par exemple, un service de messagerie sait généralement faire deux types d'actions : envoyer des mails vers l'extérieur (protocole **SMTP**<sup>4</sup>) ou recevoir des mails et les rendre disponibles à l'utilisateur via son application de messagerie (protocoles **IMAP**<sup>5</sup> ou **POP3**<sup>6</sup>). Pour le Web, le protocole s'appelle **HTTP**<sup>7</sup> ou **HTTPS** pour sa version sécurisée.

## Ports

Avec Ethernet, le réseau est un bout de câble qui relie une machine à un équipement d'aiguillage du trafic, un routeur. Il y a rarement plus d'un câble ainsi relié.

En wifi, même s'il n'y a pas de câble, le principe reste le même, il y a généralement une seule et unique liaison qui est faite entre l'ordinateur et son point d'accès au réseau (la fameuse "borne wifi"<sup>8</sup>).

On a vu que les services "échantent" en utilisant des protocoles de communication.

Tous les services utilisant donc le même bout de câble ou la même liaison wifi, comment font-ils pour savoir qui veut leur parler et comment font-ils pour ne pas se marcher sur les pieds ?

C'est pour résoudre ce problème qu'il existe des ports de communication.

Il faut voir un port comme une porte à laquelle vient frapper quelqu'un (un logiciel) qui souhaite parler à un service donné. Ainsi, chaque service étant derrière un port donné, il ne répondra qu'aux sollicitations faites sur son port.

Il y a 65535 ports disponibles sur un ordinateur en TCP/IP.

Certains ports sont réservés à un usage officiel. On peut citer, par exemple, **FTP** sur les ports **20** et **21**, **SMTP** sur le port **25**, **HTTP** sur le port **80** ou encore **HTTPS** sur le port **443**.

Certains ports sont d'un usage commun, même s'ils ne sont pas réservés officiellement.

Vous trouverez une liste de tous les ports ici :

[https://en.wikipedia.org/wiki/List\\_of\\_TCP\\_and\\_UDP\\_port\\_numbers](https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers)

Les ports > 1024 sont utilisables par n'importe quel service tournant sous n'importe quelle identité. Les ports ≤ 1024 sont réservés au système, seuls les services lancés par **root** peuvent les utiliser.

---

<sup>4</sup> Simple Mail Transfer Protocol

<sup>5</sup> Internet Message Access Protocol

<sup>6</sup> Post Office Protocol. De moins en moins utilisé au profit d'IMAP.

<sup>7</sup> HyperText Transfer Protocol

<sup>8</sup> Ainsi on dit "La borne wifi" mais on dit "le wifi" et non pas "la wifi" (c'est tonton Robert ou tata Ginette qui disent "la wifi")

## RFC

Le nom **RFC** signifie **R**equest **F**or **C**omments = Demande De Commentaires.

Les RFC sont le fondement d'Internet. Ce sont des documents qui décrivent tout ce qui fait fonctionner Internet et notamment les protocoles de communication.

Leur nom vient du fait que quand quelqu'un veut, par exemple, proposer un nouveau protocole, il va produire un document décrivant une première ébauche de sa réflexion, qu'il va soumettre à la communauté dans l'objectif de demander des commentaires (**R**equest **F**or **C**omments) sur son travail et son idée. Les retours qu'il obtiendra permettront de faire évoluer son idée vers une version qui fera consensus.

Vous trouverez la liste des RFC ici (attention, c'est lourd) : <https://www.rfc-editor.org/rfc/>

La n° **1945** est plutôt intéressante : <https://www.rfc-editor.org/rfc/rfc1945.html>, de quoi parle-t-elle ? Jetez-y un coup d'œil très rapide, on y revient plus en détail dans quelques minutes.

## Expérimentations

### Telnet

Pour tester rapidement un protocole, une bonne vieille méthode de barbu consiste à utiliser **Telnet**.

**Telnet** est le nom de 2 choses : un protocole et un programme (qui utilise ce protocole).

Le protocole **Telnet** est on ne peut plus primaire. Il reçoit et envoie du texte brut. Ce texte peut éventuellement contenir quelques séquences spéciales de caractères permettant d'afficher des attributs graphiques comme du gras, de l'italique ou de la couleur de texte ou d'arrière-plan du texte. C'est hyper basique comme fonctionnement. A la fin de ce TP, vous serez capable d'écrire un programme qui exploite **Telnet**, comme un barbu de 1969 (date de l'invention de Telnet).

Le programme **telnet** permet, en exploitant ce protocole, d'envoyer à un autre **telnet** du texte et, réciproquement, d'afficher<sup>9</sup> ce qu'il reçoit..

Mais que viennent faire ces **[Tt]elnet** dans cette histoire ?

Du fait de son caractère très basique fonctionnant par "j'envoie du texte et j'affiche ce que je reçois", **telnet** va nous permettre de tester manuellement des protocoles, à condition de connaître leur fonctionnement. "Manuellement" signifie : sans avoir besoin d'un quelconque logiciel client sachant "parler le protocole" qu'on souhaite tester. Évidemment on pourrait utiliser des logiciels clients dédiés, mais ça nous masquerait complètement les messages qui sont échangés !

---

<sup>9</sup> Plus précisément, il l'envoie sur son **STDOUT**, qui est, par défaut, l'écran.

Nous allons expérimenter cela immédiatement avec un protocole que vous utilisez tous les jours : **HTTP**.

## HTTP - Protocole Web

Vous devez avoir trouvé que la RFC n° **1945** est celle qui a introduit le protocole **HTTP**.

On y trouve le nom de **Tim Berners-Lee**, le papa de HTTP. Cette RFC est un bout d'histoire de notre célèbre réseau mondial puisque c'est elle et les commentaires reçus en retour qui ont fixé les fondements du Web.

Depuis cette version originale (1990-96), plus d'une douzaine d'autres RFC sont venues amender le protocole HTTP. On y découvre le mécanisme de ce protocole.

Ce document fait une soixantaine de pages et est assez limpide dans ses explications. Si vous aviez à concevoir et développer un navigateur Web, par exemple, ce serait une source d'information essentielle, conjointement aux RFC 2068, 2616, 7230 à 7237 et 7540 ! Je vous rassure, pour ce TP nous n'allons pas le regarder en détail mais juste jeter un œil à quelques passages pour comprendre un peu le protocole HTTP car nous allons devoir jouer avec en nous passant d'un navigateur Web.

NB : je me réfère donc à la **RFC 1945** officielle, au format HTML uniquement (car il contient des liens pour faciliter la navigation dans le document). Si vous n'arrivez pas à y accéder, vous en trouverez une copie avec le sujet sur Moodle.

### Requêtes

Comme beaucoup de protocoles, le HTTP fonctionne sur un principe de "requêtes du client, réponse du serveur". Ce protocole est ainsi fait, le serveur n'émet jamais de requête vers le client. Mais d'autres protocoles peuvent avoir un fonctionnement bi-directionnel.

En HTTP, effectuer une requête est simplement le fait de demander au serveur Web une page HTML ou un élément constitutif de la page (images, CSS, JS, etc).

Le client Web, généralement un navigateur, commence par demander la page HTML au serveur (1<sup>ère</sup> requête), puis il l'analyse et la décortique pour en extraire la liste des éléments constitutifs (**<img src=...>**, **<link href=...>**, etc), pour lesquels il va faire autant d'autres requêtes qu'il y a d'éléments, afin de reconstruire le puzzle de la page complète, pour finalement pouvoir afficher le résultat final.

Ainsi, si une page HTML contient 10 images, 1 lien vers un fichier CSS et 1 lien vers un fichier JS, le navigateur fera donc 13 requêtes au total. Pour optimiser le temps, il en fera certaines en parallèle, si possible (images, CSS, JS).

### Méthodes

Il existe plusieurs méthodes pour envoyer une requête (rien à voir avec la BDD) à un serveur Web, en fonction de l'action qu'on souhaite faire.

Vous en connaissez certainement déjà deux, qu'on trouve en HTML dans l'attribut **METHOD** d'un tag **<FORM>**. Il s'agit de **GET** et de **POST**.

Sans entrer dans les détails, un **POST** est généré lié à l'envoi de grosses quantités de données depuis le formulaire, comme par exemple quand on envoie des fichiers, sinon on utilise souvent **GET**.

**GET** et **POST** sont donc des méthodes de requête.

Il en existe une troisième : **HEAD**, on y revient plus loin.

## GET

Nous allons regarder ce que nous dit la documentation du protocole HTTP pour une requête faite en méthode **GET**.

Rendez-vous aux 1<sup>ère</sup> et 2<sup>nde</sup> pages du document, vous y trouvez la table des matières (cliquable si vous êtes sur le document HTML).

Le chapitre 5 parle des requêtes et on y apprend ceci<sup>10</sup> :

**...there are two valid formats for an HTTP request:**

**Request** = **Simple-Request** | **Full-Request**

**Simple-Request** = **"GET"** **SP** **Request-URI** **CRLF**

**Full-Request** = **Request-Line**  
\*( **General-Header**  
| **Request-Header**  
| **Entity-Header** )  
**CRLF**  
[ **Entity-Body** ]

C'est ce qu'on appelle un élément de grammaire. Vous connaissez<sup>11</sup> la grammaire française qui permet de décrire comment on constitue des phrases intelligibles. Un exemple simpliste pourrait être qu'une phrase est construite avec un **SUJET** suivi d'un **VERBE** suivi d'un **COMPLEMENT** optionnel. En informatique, pour les langages par exemple, ou encore pour les protocoles, on définit aussi des grammaires qui décrivent comment les instructions ou les blocs d'instructions doivent être formatés pour être valides.

Pour une requête HTTP, on apprend donc, avec cet élément de grammaire issu de la RFC, qu'elle peut avoir 2 formes (c'est le **|** qui est un "ou" et qui donne donc plusieurs alternatives) : soit une forme **Simple-Request**, soit une forme **Full-Request**.

Puis, la grammaire définit les éléments qui n'ont pas encore été définis à ce stade du document, à savoir **Simple-Request** et **Full-Request**.

Un **Simple-Request** a une seule forme possible : un **"GET"** suivi d'un **SP** suivi d'un **Request-URI** suivi d'un **CRLF**.

<sup>10</sup> Vous devez aller voir dans le document pour vérifier ce que je dis 😊

<sup>11</sup> En principe, même si parfois on a des doutes pour certain.e.s

Il a été défini en page 9 que les éléments entre `""` sont des littéraux, c'est-à-dire le texte brut présent entre les `""` mais sans les `""`. Donc `"GET"` est simple le texte `GET`.

Il a été défini en page 11 que `CRLF` signifie un `CR` suivi d'un `LF` et un peu plus haut dans cette page 11 ont été définis `CR` et `LF` qui signifient respectivement `carriage return` (la touche `ENTRÉE`, code ASCII 13, un `\r` en C) et `line feed` (code ASCII 10, un `\n` en C). Donc `CRLF` sera `\r\n`.

On a presque tout ce qu'on doit savoir pour faire un `GET`, il manque juste une définition de ce qu'est un `Request-URI`. On trouve sa définition page 24, à la section 5.1.2.

On y apprend qu'un `Request-URI` est simplement un chemin absolu (du point de vue du serveur Web) vers la ressource qu'on souhaite accéder. C'est la fameuse `URL`.

Note : on s'arrêtera là, on ne va pas parler du format d'un `Full-Request`.

Tout ce qu'on vient de décrire ensemble est un exemple de la façon dont sont construites les RFC.

Nous n'allons pas aller plus loin dans le détail du HTTP, ce n'est pas l'objet de ce TP. Cependant, nous allons mettre en pratique ce que nous venons de découvrir sur le format d'un requête `GET`.

## Test d'un GET avec `telnet`

Revenons à notre ami le programme `telnet` qui va nous être utile pour tester le protocole HTTP avec une requête `GET` dont on connaît maintenant la syntaxe.

`telnet` va nous servir de client HTTP, à la place d'un navigateur.

Mais pour tester il nous faut un programme qui sera capable de répondre à nos requêtes HTTP, c'est-à-dire un serveur Web. Il se trouve que le programme `php` lui-même intègre un mini serveur Web quand on le lance avec une option spéciale.

Voici les étapes préparatoires :

- Ouvrez un Terminal
- Créez un dossier de votre choix
- Déplacez-vous dans ce dossier
- Créez un fichier `doc.html` avec le contenu de votre choix, faites simple.
- Lancez un :

```
| php -S localhost:8080
```

(attention, c'est un **S** majuscule)

- Vérifiez immédiatement dans un navigateur que <http://localhost:8080/doc.html> affiche bien votre page.

Notez que le port Web est généralement le port **80** mais il ne vous est pas possible d'utiliser ce port sur vos machines de l'IUT car c'est un port réservé et il faudrait être **root** pour pouvoir lancer le `php -S` sur le port **80**, c'est pour cela qu'on a choisi le port **8080** à la place et qu'on doit le spécifier aussi dans l'URL du navigateur.

---

Maintenant nous allons causer “la langue HTTP” comme des barbus, grâce à **telnet** :

- Ouvrez un second Terminal
- Lancez un :

```
telnet localhost 8080
```

(respectez bien les espaces)

- Ne portez pas attention aux trois lignes qui s’affichent. Ces lignes n’ont rien à voir avec le protocole HTTP, c’est juste **telnet** qui nous accueille en précisant qu’il a bien réussi à établir une connexion.

Notez qu’on a passé 2 paramètres à **telnet** pour qu’il établisse une connexion au serveur : une adresse IP qui est **127.0.0.1** (c’est l’IP derrière le nom **localhost**) et un port (ici **8080**).

A ce stade, vous avez donc réussi à établir une connexion avec un serveur Web se trouvant à l’adresse IP **127.0.0.1**, et en utilisant le port **8080**.

Le programme **telnet** est maintenant en attente de saisies au clavier qu’il va envoyer au serveur, par la connexion réseau établie. Il est en même temps dans l’attente de ce que le serveur peut lui envoyer sur cette même connexion. Il est dans cet état jusqu’à ce que la connexion soit refermée par le serveur, signifiant ainsi que la partie est terminée !

On y est presque, on va pouvoir mettre en pratique notre connaissance de la grammaire **HTTPIène** qu’on maîtrise désormais pour le **GET**. Tapez ceci dans le programme **telnet** :

```
GET /doc.html
```

<- une ligne vierge (par appui sur ENTRÉE)

Notez bien cette ligne vierge qu’il faut saisir en tapant une seconde fois la touche **ENTRÉE**.

On vérifie ainsi qu’on respecte bien la syntaxe d’un **GET** avec une **Simple-Request**, comme on l’a appris précédemment, et que cette requête doit être terminée par **CRLF**, donc 2 fois la touche **ENTRÉE**, une pour le **CR** et une pour le **LF** (oui ces 2 codes sont produits par la même touche dans **telnet**).

Et la magie opère ! Vous devriez obtenir un affichage du code source de la page **doc.html** !

Vous noterez aussi que la page HTML est précédée de plusieurs lignes qu’on appelle l’en-tête (rien à voir avec la section **<HEAD>** dans le code HTML) et que ces lignes sont, elles aussi, séparées du code HTML par une ligne vierge.

Le protocole HTTP a fait ce choix qu’on retrouve aussi dans d’autres protocoles :

- Une ligne vierge permet, lors de l’envoi de notre requête, d’indiquer que la requête est complète (car une requête peut tenir sur plusieurs lignes, comme on le verrait si on étudiait complètement le protocole HTTP).



- Lors de la réception de la réponse, la ligne vide permet cette fois-ci de savoir où s'arrête l'en-tête (qui contient plein d'informations pratiques, notamment les cookies) et où commence réellement le code HTML qui est ce que le navigateur aura à afficher.

Notez enfin que **telnet** a quitté car le serveur a refermé la connexion après avoir envoyé sa réponse, signifiant ainsi que la requête est répondue et qu'il n'y a rien d'autre qui arrivera de lui.

Dans le cas d'un navigateur Web, il décortiquera ensuite le contenu du code HTML reçu et fera autant de nouvelles requêtes que d'éléments complémentaires présents dans la page HTML. Ainsi : 1 requête = 1 connexion = 1 élément récupéré.

Pour terminer, regardez la partie correspondant à l'en-tête et observez les informations intéressantes qu'elle contient : date de la page, type de contenu, taille, ainsi que la version du protocole HTTP que le serveur utilise pour dialoguer.

Dans cet en-tête vous trouvez aussi le code **200 OK** qui indique que ça c'est bien passé.

Si vous tentez de faire un **GET /blabla** on obtient le fameux **404 Not Found**.

Le protocole renvoie de cette façon un code facilement identifiable pour indiquer si la requête est valide ou pas.

## Sockets

Nous allons maintenant passer du stade expérimental à la phase de codage des sockets Unix.

Le socket est le nom du mécanisme qui permet d'établir une connexion réseau TCP/IP. Les sockets peuvent être utilisés avec d'autres protocoles que TCP/IP mais nous n'allons les étudier que dans le cadre de TCP/IP.

Il y a plusieurs fonctions système à connaître pour utiliser les sockets.

Il faut aussi se rappeler (ou découvrir si vous n'aviez pas encore "percuté") qu'une connexion réseau fait entrer en jeu 2 protagonistes, qui peuvent avoir des rôles identiques mais qui sont souvent un client pour l'un et un serveur pour l'autre.

Peu importe le rôle de l'un et de l'autre, pour avoir une communication par socket réseau, il faut impérativement que l'un se mette en écoute (attente) que quelqu'un d'autre frappe à la porte (au port), et que l'autre vienne frapper à la porte.

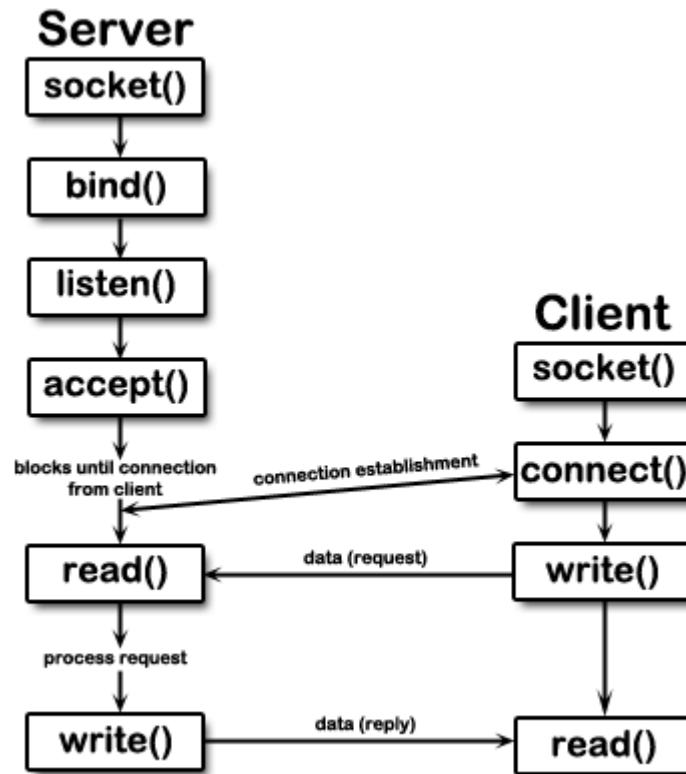
Il est impossible d'établir une connexion entre deux parties qui seraient toutes deux en attente ou si aucune des deux n'est à l'écoute/en attente.

Les voici dans l'ordre d'apparition dans tout code qui fait appel aux sockets.

Pour voir les **#include** nécessaires, allez dans le **man** de chacune des fonctions qui vont vous être présentées car ils ne sont pas indiqués dans les codes ci-après. Consultez aussi le **man inet** qui est une source d'information pour les **#include**.

Avant de commencer, voici déjà un schéma de la chronologie des appels systèmes en fonction de la situation (client ou serveur).

Gardez un œil sur ce schéma au fur et à mesure de notre progression :



(source : <https://www.cs.uregina.ca/Links/class-info/330/Sockets/sockets.html>)

## Fonction `socket()` - Client et Serveur

A l'instar d'un descripteur de fichier qu'on obtient à l'aide d'un `open()` et qui permet de faire ensuite des actions<sup>12</sup> sur le fichier ouvert, il faut aussi un descripteur de socket pour faire des actions sur et avec un socket.

Pour obtenir un tel descripteur, qu'on appelle souvent un **handler**<sup>13</sup>, on fait l'appel système suivant :

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int sock;
```

```
sock = socket(AF_INET, SOCK_STREAM, 0);
```

<sup>12</sup> `read()`, `write()`, `close()`, etc.

<sup>13</sup> To handle = manipuler, a handle = une poignée, a handler = un "manipulateur"

Nous n'entrerons pas dans l'explication des 3 paramètres passés à `socket()`, pour nos besoins de cette année, ce seront toujours ces 3 valeurs. La documentation est là pour les plus curieux ou si vous avez d'autres besoins, mais ce jour-là vous le saurez certainement.

Attention, avoir un **handler** de socket ne signifie pas qu'on a établi une connexion avec qui que ce soit. On a juste préparé le terrain mais il reste des actions à faire avec les fonctions qui suivent.

## Fonction `bind()` - Serveur seulement

Il nous faut maintenant configurer le socket pour indiquer au système sur quel port on souhaite communiquer et sur quelle interface réseau<sup>14</sup>.

C'est le rôle de la fonction système **`bind()`**.

Pour information, même si c'est une expression trompeuse, cette action de **`bind()`** s'appelle "nommer un socket". Par nommage on entend en fait lui donner une IP et un port.

Voici l'appel à effectuer et il suppose que l'appel à **`socket()`** a déjà été fait en amont, bien entendu :

```
#include <sys/types.h>
#include <sys/socket.h>

int sock;
int ret;
struct sockaddr_in addr;

sock = socket(AF_INET, SOCK_STREAM, 0);
addr.sin_addr.s_addr = inet_addr("127.0.0.1");
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
ret = bind(sock, (struct sockaddr *)&addr, sizeof(addr));
```

Vous aurez observé qu'on définit une variable **`addr`** de type **`struct sockaddr_in`** pour l'utiliser dans un contexte où on attend un pointeur de type **`struct sockaddr`**, d'où le cast du 2<sup>ème</sup> argument de **`bind()`**. Même si elles sont légèrement différentes, ces deux types de structures sont compatibles. Nous n'expliquerons pas en détail cette bizarrerie qui n'a pas grande importance à notre niveau pour ce TP.

---

<sup>14</sup> Un ordinateur peut avoir plusieurs interfaces réseau. On a souvent une carte réseau physique mais il peut aussi exister des cartes virtuelles, comme notamment la fameuse **`localhost`** (IP **`127.0.0.1`**) ou encore des interfaces VPN par exemple.

Notez que la variable **addr.sin\_addr.s\_addr** est affectée par l'IP **127.0.0.1** ce qui signifie que ce socket sera utilisé uniquement sur cette IP (**localhost**). Il est possible d'utiliser un socket sur toutes les interfaces disponibles sur la machine en la remplaçant par la ligne suivante :

```
addr.sin_addr.s_addr = INADDR_ANY;
```

Pour le moment, on va se limiter à **localhost**.

A ce stade, le socket est configuré avec une IP et un port mais il n'est pas encore à l'écoute derrière "la porte". C'est ce que nous allons mettre en place à l'étape suivante.

## Fonction listen() - Serveur seulement

Cette fonction système permet au processus de se mettre à l'écoute, sur son port, que des clients se présentent et viennent "frapper à la porte".

Voici la syntaxe (certaines lignes du code précédent sont volontairement absentes pour un gain de place ci-dessous, ne les supprimez pas du vôtre !) :

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int sock;
int ret;

ret = listen(sock, 1);
```

Le second paramètre (ici à la valeur **1**) indique la taille de la queue de connexion, une sorte de file d'attente.

Comme on va le voir avec la fonction système suivante (**accept()**), quand le processus reçoit une connexion, il se retrouve ensuite dans une succession d'échanges entre lui et le client (ce sont les messages échangés tels que décrits par le protocole). Durant cette période d'échanges qui, en fonction du protocole, peut être longue, le processus ne pourra plus recevoir de nouvelle connexion et si un autre client se présente, il sera alors rejeté. Ce second paramètre de **bind()** permet d'indiquer au système le nombre de clients qui peuvent se présenter et être mis en file d'attente. Quand la file est pleine, les nouvelles connexions sont rejetées.

Gardons-le à **1** pour nos tests.

Le nom de cette fonction est trompeur car ce n'est pas elle qui met le processus en attente de l'arrivée d'un client. On va leur voir, c'est la fonction suivante qui a ce rôle.

Nous sommes désormais à un appel système d'avoir notre premier serveur opérationnel.

## Fonction `accept()` - Serveur seulement

La fonction **`accept()`** est bloquante et met le processus en attente qu'un client se présente au port (ici le **8080**) sur l'interface réseau spécifiée dans **`bind()`** (**localhost** ici).

Elle rend la main lorsqu'un client a "frappé à la porte".

En voici la syntaxe :

```
#include <sys/types.h>
#include <sys/socket.h>

int sock;
int ret;
int size;
int cnx;
struct sockaddr_in conn_addr;
size = sizeof(conn_addr);
cnx = accept(sock, (struct sockaddr *)&conn_addr, (socklen_t *)&size);
```

Le troisième argument peut paraître un peu étrange car il sert à passer la longueur de la **struct sockaddr** mais on passe l'adresse d'une variable contenant cette taille et non pas la taille elle-même. La raison est qu'en retour on a besoin de récupérer la taille de cette même structure mais concernant cette fois-ci le client. C'est donc un argument servant en double sens.

La fonction accepte renvoie, en cas de succès, un descripteur de connexion, au même titre qu'un appel à **`open()`** renvoie un descripteur de fichier. C'est ensuite grâce à ce descripteur que le processus va lire et écrire les données depuis et vers le client, comme on lirait et écrirait depuis et vers un banal fichier, jusqu'à sa fermeture.

Notez que, pour les exercices suivants, il est possible que vous ayez une erreur **Address already in use** lors d'un **`bind()`**.

Cela arrive soit parce que le port est déjà ouvert par un autre processus<sup>15</sup>, soit plus probablement parce que vous avez relancé votre processus trop rapidement après l'avoir interrompu par un **CTRL+C** par exemple. Il est alors possible que le port soit encore considéré ouvert par le système.

<sup>15</sup> Dans ce cas arrêtez ce processus ou changez de port.

Dans ce cas attendez quelques secondes ou choisissez un autre port, mais ça nécessite une recompilation qui prendra sans doute autant de temps qu'attendre les 15-20 secondes qui libéreront le port.

## Exercice 1

Mettez en œuvre ce premier serveur. Le code est en principe complet si vous avez suivi pas à pas. Ajoutez-y quelques **printf()** pour détailler ce qu'il fait, que les valeurs de retour des fonctions système sont bonnes, pour afficher éventuellement les erreurs et enfin pour vérifier qu'il est bien arrivé en attente d'une connexion.

Avant d'exécuter votre programme, prenez soin de quitter le **php -S** précédent car il utilise déjà le port **8080** sur **localhost**, ce qui empêchera votre programme de pouvoir exploiter le même port sur la même interface, c'est logique.

Une fois votre programme compilé, lancé et bloqué en attente testez-le avec un :

```
telnet localhost 8080
```

Cette commande devrait mettre un terme immédiat à votre processus car il va recevoir une connexion mais rien n'est prévu (en principe) pour faire quoi que ce soit de cette connexion et votre programme doit s'arrêter en mettant donc un terme à la connexion.

C'est un bon début et nous allons évidemment faire mieux maintenant.

## Fonctions **read()** et **recv()** - Client et Serveur

On connaît la fonction **read()** qui est une fonction système permettant de lire dans un fichier. On a aussi vu que, sous Unix, beaucoup de choses sont considérées comme des fichiers : les fichiers standards, évidemment, mais aussi les fichiers spéciaux tels que les tubes (**pipe()**). Il en existe d'autres et les sockets sont aussi considérés un peu comme des fichiers spéciaux. Ainsi **read()** permet d'y lire des données. C'est très pratique.

Cependant, même si un flux de données arrivant sur un socket peut être considéré comme un fichier, ce flux a des caractéristiques et un mode de fonctionnement différents d'un vrai fichier. Par exemple, on ne peut pas se déplacer où on veut en avant ou en arrière comme on le ferait avec un fichier : un flux est linéaire, consécutif et a une temporalité qu'on ne trouve pas avec un fichier.

De ce fait, il existe une autre fonction permettant de lire sur un socket et offrant des options spécifiques aux connexions réseau. Cette fonction système s'appelle **recv()**<sup>16</sup>.

Si vous n'avez pas besoin de ces options, un **read()** fait très bien l'affaire, sinon vous utiliserez **recv()**. Pour le moment on va se satisfaire de **read()**.

Comme avec les tubes, la lecture dans un socket a 3 valeurs de retour possibles :

---

<sup>16</sup> **recv** signifie **receive**

- **> 0** : des données ont été lues et cette valeur indique le nombre d'octets lus.
- **== 0** : rien n'a été lu, le socket a été fermé par l'autre côté (le client ou le serveur, en fonction de la situation)
- **== -1** : seule valeur négative possible. Elle indique une erreur ou une situation que `errno` explicitera. Lire `man` pour plus de détails. Lire notamment l'encadré en fin de TP.

Les lectures doivent donc se faire en boucle tant que **read()** ne renvoie pas **0** signifiant une fermeture du socket, ou tant qu'on n'a pas soi-même décidé de mettre un terme à l'échange.

## Fonctions **write()** et **send()** - Client et Serveur

On connaît aussi la fonction **write()** qui est une fonction système permettant d'écrire dans un fichier. A l'instar de **read()**, on peut utiliser **write()** sur les sockets et on dispose aussi, pour les mêmes raisons que celles expliquées précédemment, d'une fonction spécifique aux connexions réseau, qui s'appelle **send()**.

### Exercice 2

Complétez votre exercice 1 en y ajoutant la lecture d'un bloc de texte (une seule lecture, pas de boucle). Ajoutez des logs (**printf()**) pour expliquer ce qui se passe. Compilez, exécutez et testez avec un **telnet** dans un second terminal.

### Exercice 3

Complétez votre exercice 2 en y ajoutant l'écriture d'un bloc de texte après la lecture. Ajoutez des logs (**printf()**) pour expliquer ce qui se passe. Compilez, exécutez et testez avec un **telnet** dans un second terminal, et vérifiez que **telnet** affiche bien ce qui lui vient du serveur.

### Exercice 4

Écrivez un programme serveur qui répond en boucle au protocole suivant :

- Renvoie **COUCOU LES GENS** sur réception du message **HELLO** (terminé par un **\r**)
- Renvoie **PONG N°X** sur réception du message **PING** (terminé par un **\r**) en indiquant le nombre **X** de fois où ce type de message est affiché.
- Renvoie le résultat du calcul sur réception d'un message du type **MMM\*NNN** (terminé par un **\r**) où **MMM** et **NNN** sont des valeurs numériques sous forme de texte.
- Termine la connexion sur réception du message **BYE BYE** (terminé par un **\r**)

Le choix du **\r** ci-dessus est qu'en testant avec **telnet** la touche **ENTRÉE** enverra un **\r\n**, donc en testant juste la présence du **\r** ce sera suffisant. Faites un **man strncmp**, ça peut vous servir.

Testez encore avec un **telnet**.

## Fonction connect() - Client

Cette fonction ne concerne que les programmes clients qui vont donc se connecter à un serveur. Une fois le socket obtenu avec un appel à **socket()**, il ne reste plus qu'à effectuer la connexion pour commencer les échanges avec **read()** et **write()**. Voici le code de base qui ressemble comme deux gouttes d'eau au **bind()** du côté serveur mais qui, de ce côté client, permet d'établir immédiatement une connexion :

```
#include <sys/types.h>
#include <sys/socket.h>

int sock;
int ret;
struct sockaddr_in addr;

sock = socket(AF_INET, SOCK_STREAM, 0);
addr.sin_addr.s_addr = inet_addr("127.0.0.1");
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
ret = connect(sock, (struct sockaddr *)&addr, sizeof(addr));
```

Une fois la connexion établie (et qu'on a vérifié que c'est OK, voir encadré ci-dessous), on peut faire des **read()** et **write()** jusqu'à ce que l'un ou l'autre renvoie un code indiquant que le socket est fermé ou qu'on décide de le fermer nous-mêmes.

### Petite subtilité importante

Sur le client, les **read()** et **write()** se font en utilisant le descripteur retourné par la fonction **socket()** (i.e. la variable **sock**).

Sur le serveur, les **read()** et **write()** se font en utilisant le descripteur retourné par la fonction **accept()** (i.e. la variable **cnx**).

Rappel : dans toutes ces fonctions, vous aurez noté qu'on récupère un code de retour (placé ici dans la variable **ret**) qu'il convient donc de tester pour détecter un problème. Consultez le **man** de chacune d'elles pour plus d'informations.

Comme pour toutes les fonctions système, vous en avez maintenant l'habitude, la variable **errno** et la fonction **perror()** sont vos amies.



Notez aussi qu'une fonction bloquante comme **accept()** peut rendre la main sans que ça signifie qu'une connexion est arrivée ! Si un signal survient et que vous avez attrapé ce signal, au retour du traitement du signal l'exécution se poursuivra **APRÈS** (c'est-à-dire en retour de) **accept()**, qui retourne alors un **-1** et **errno** est placé à **EINTR** qui signifie que l'appel a été interrompu (regardez le **man accept**). Ce n'est pas vraiment une erreur, c'est une indication que vous devez sans doute repartir faire un **accept()** car aucun client n'est réellement arrivé à la porte !